03 / LEARNING GUIDE

EX28 — Exercise Guide

Matrices and matrix operations · Class 18

> **Reading order:** 01 Engineering & Math Theory → 02 Python Reference → 03 Exercise → 04 Pseudocode → programming. This guide belongs to EX28, class 18.

1. Objective and prerequisites

The objective is to translate basic matrix algebra into a simple Python program. We reuse arrays and norms from EX25, dot products from EX26, and vector components from EX27. We do not yet solve systems of unknown variables.

By the end, you should be able to: identify compatible dimensions, transpose matrices, add them, distinguish * from @, multiply a matrix by a vector, and interpret a physical result.

2. Working folder

```
Python_Engineering/
└── 02_numerical_methods/
    └── algebra/
        ├── class_17_vectors/
        │   └── EX27_cross_product.py
        └── class_18_matrices/                ← CURRENT CLASS
            ├── EX28_matrix_operations.py     ← CREATE
            └── docs/
                ├── ES/
                └── EN/
```

Create only `EX28_matrix_operations.py`. Keep the guides in `docs/`. No data files, JSON, `src/` folders, or automated tests are required.

3. Part A0 — Fully worked algebra example

Study this example **before writing Python**. Begin with the following matrix and vector:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad \boldsymbol{x} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}$$

Step 1: transpose A by exchanging rows and columns.

$$A^T = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

Step 2: compute A times x using the dot product of each row with the vector.

$$A\boldsymbol{x} = \begin{pmatrix} 1(2) + 2(-1) \\ 3(2) + 4(-1) \end{pmatrix} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$$

A has shape (2,2), x has shape (2,), and the result has shape (2,). Verify both entries by hand.

4. Part A1 — Your matrix exercise

Use the following arrays and perform the calculations **independently**. The check values let you verify your answer; they are not a replacement for the calculations.

$$C = \begin{pmatrix} 2 & -1 \\ 0 & 3 \end{pmatrix}, \quad D = \begin{pmatrix} 1 & 4 \\ -2 & 2 \end{pmatrix}, \quad \boldsymbol{x} = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$$

- **A1.1** Create C, D and x in NumPy. Print each array's `shape` and `ndim`.
- **A1.2** Calculate C + D and C^T.
- **A1.3** Calculate C**x** and verify both entries by hand.
- **A1.4** Calculate CD and DC; compare the results.
- **A1.5** Compute element-wise C * D. In one sentence, explain why it differs from CD.

A1 reference results

$$C + D = \begin{pmatrix} 3 & 3 \\ -2 & 5 \end{pmatrix}, \quad C^T = \begin{pmatrix} 2 & 0 \\ -1 & 3 \end{pmatrix}$$

$$C\boldsymbol{x} = \begin{pmatrix} 4 \\ 6 \end{pmatrix}, \qquad CD = \begin{pmatrix} 4 & 6 \\ -6 & 6 \end{pmatrix}$$

$$DC = \begin{pmatrix} 2 & 11 \\ -4 & 8 \end{pmatrix}, \qquad C * D = \begin{pmatrix} 2 & -4 \\ 0 & 6 \end{pmatrix}$$

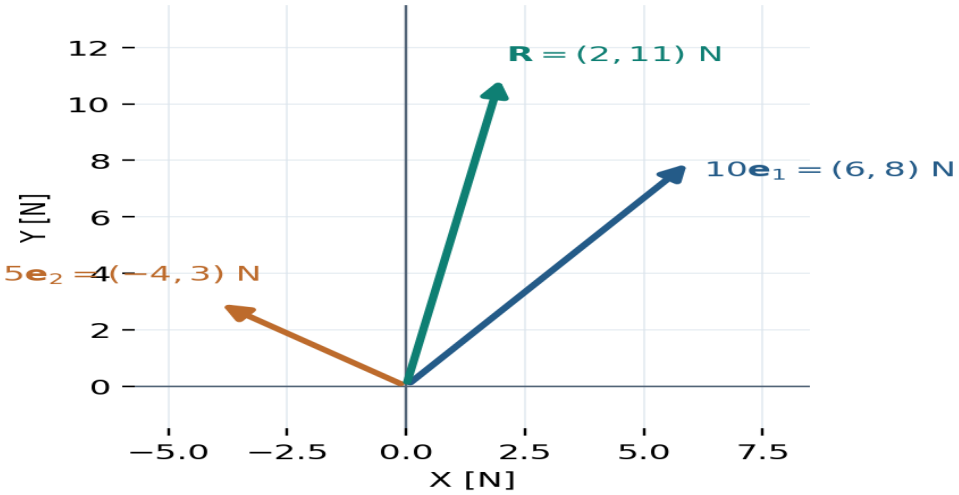
5. Part B — Engineering: resultant of known forces

Two actuators apply forces in the XY plane. Their magnitudes are **10 N** and **5 N** in the following unit directions:

$$\boldsymbol{e}_1 = \begin{pmatrix} 0.6 \\ 0.8 \end{pmatrix}, \qquad \boldsymbol{e}_2 = \begin{pmatrix} -0.8 \\ 0.6 \end{pmatrix}$$

Matrix A holds these directions as its columns. Vector **u** contains the force magnitudes:

$$A = \begin{pmatrix} 0.6 & -0.8 \\ 0.8 & 0.6 \end{pmatrix}, \quad \boldsymbol{u} = \begin{pmatrix} 10 \\ 5 \end{pmatrix} \text{N}$$



- **B1.** Define A and u as arrays and display their dimensions.

- **B2.** Calculate the resultant force vector using A @ u.
- **B3.** Independently calculate $10e_1 + 5e_2$ and compare.
- **B4.** Find the norm 2 of the resultant using your EX25 knowledge.
- **B5.** Interpret the rows, columns and physical units of A, u and R.

Part B reference results

$$\boldsymbol{R} = \begin{pmatrix} 2 \\ 11 \end{pmatrix} \mathrm{N}, \qquad \|\boldsymbol{R}\|_2 = \sqrt{125}\,\mathrm{N} \approx 11.1803\,\mathrm{N}$$

Check manually: $R_x = 0.6 \times 10 - 0.8 \times 5$ and $R_y = 0.8 \times 10 + 0.6 \times 5$. Do not use `np.linalg.solve`; both force magnitudes are already given.

6. Suggested script organization

You may divide your script into **BLOCK A0**, **BLOCK A1** and **BLOCK B**. Define input data, calculate intermediate results, then display them. Print units for physical results only.

7. Python hints (at the end, not the full solution)

```
import numpy as np

# Example 2D matrix
M = np.array([[1, 0], [0, 1]], dtype=float)
print(M.shape)    # (2, 2)
print(M.T)        # Transpose

# M * N and M @ N do not mean the same thing.
```

Use guide 04 to design your pseudocode. For a dimension-related error, inspect the `shape` of each operand first.